ARCHON

AI Software Archaeologist & Self-Healing Legacy Code Platform

ULTIMATE MASTER ENGINEERING PROMPT

You are the **lead software architect, senior backend engineer, AI engineer, software-analysis engineer, QA engineer, security engineer, and DevOps engineer** responsible for designing and building **ARCHON from scratch**.

This is a **GREENFIELD SOFTWARE PROJECT**.

There is **NO existing ARCHON codebase** to audit, extend, or preserve.

You are authorized to create and modify the project files directly.

Your responsibility is to transform this specification into a **real, working, tested, integrated software engineering platform**.

This is NOT a request to build:

- a presentation
- a demo
- a mockup
- a fake prototype
- a project report
- a viva project shell
- a UI-only application
- disconnected AI agents
- hardcoded analysis
- simulated test results

You are building the **actual software product**.

---

1. PROJECT VISION

ARCHON

AI Software Archaeologist & Self-Healing Legacy Code Platform

ARCHON is an AI-powered software engineering platform that accepts an unfamiliar or legacy Git/GitHub repository and progressively builds an evidence-backed understanding of the software.

ARCHON should help engineers answer:

What does this repository do?

How is it structured?

What are its major components?

How do those components depend on each other?

Why does this legacy code exist?

How has this code evolved?

What assumptions are hidden in the implementation?

Which components are risky?

What could break if this component changes?

Which behavior should be preserved?

Where are the testing gaps?

Why is something failing?

What is the probable root cause?

Can the problem be repaired safely?

Which repair candidate is safest?

Was the repair actually verified?

How can the repaired behavior be protected against regression?

What should be modernized?

What is the safest modernization order?

ARCHON combines:

Repository Intelligence

Source-Code Analysis

Architecture Reconstruction

Dependency Analysis

Git Archaeology

Behavior Reconstruction

Hidden Assumption Detection

Legacy DNA

Technical Debt Analysis

Risk Analysis

Change Safety

Change Impact Simulation

Characterization Testing

AI Test Generation

Failure Investigation

Root Cause Analysis

Self-Healing

Patch Verification

Regression Protection

Incident Memory

Repository Comparison

Modernization Intelligence

GitHub Integration

REST API

Frontend

Excel Reporting

These are NOT independent modules.

They must form one integrated engineering system.

---

2. CORE ARCHITECTURAL PRINCIPLE

ARCHON must be **evidence-driven**.

The fundamental pipeline is:

REAL REPOSITORY

↓

DETERMINISTIC ANALYSIS

↓

REPOSITORY KNOWLEDGE

↓

EVIDENCE

↓

AI REASONING

↓

ENGINEERING DECISION

↓

TEST / EXECUTION

↓

VERIFICATION

AI must not replace deterministic analysis where deterministic analysis is possible.

Use deterministic engineering for:

AST Parsing

Git Analysis

Complexity

Churn

Dependency Analysis

Graph Construction

Coverage

Test Execution

Patch Application

Patch Verification

Use AI where reasoning genuinely adds value:

Historical Intent

Behavior Interpretation

"Why Does This Exist?"

Hidden Assumption Interpretation

Root Cause Hypotheses

Test Scenario Generation

Patch Generation

Modernization Strategy

Documentation

---

3. HARD MVP

The MVP is the complete closed loop below.

ARCHON must be capable of taking a **real supported GitHub repository** and executing:

REAL GITHUB REPOSITORY

↓

VALIDATION

↓

SECURE CLONE

↓

REPOSITORY SNAPSHOT

↓

SOURCE-CODE ANALYSIS

↓

DEPENDENCY ANALYSIS

↓

GIT HISTORY ANALYSIS

↓

ARCHITECTURE RECONSTRUCTION

↓

LEGACY RISK ANALYSIS

↓

CHANGE SAFETY ANALYSIS

↓

TEST ANALYSIS

↓

CHARACTERIZATION / TEST GENERATION

↓

SECURE TEST EXECUTION

↓

FAILURE DETECTION

↓

AI ROOT CAUSE INVESTIGATION

↓

CANDIDATE PATCH GENERATION

↓

SANDBOX PATCH APPLICATION

↓

PATCH VERIFICATION

↓

REGRESSION VERIFICATION

↓

VERIFIED / REJECTED

This is the **non-negotiable MVP**.

Do not allow secondary features to consume significant engineering effort until this loop works.

---

4. MVP SUCCESS CRITERIA

The MVP must genuinely support:

Repository

- GitHub URL ingestion
- repository validation
- repository metadata
- branch selection
- commit selection
- secure cloning
- snapshot creation
- commit SHA tracking

Source Intelligence

Files

Modules

Classes

Functions

Methods

Imports

Calls

Dependencies

Complexity

Tests

Configuration

Entry Points

Git Intelligence

Commits

Commit Messages

Changed Files

Additions

Deletions

Churn

Change Frequency

Component Age

Co-changing Files

Architecture

Components

Dependencies

Callers

Dependents

Architecture Roles

Entry Points

Legacy Intelligence

Legacy DNA

Technical Debt

Hotspots

Repository Understanding

Legacy Risk

Change Safety

Testing

Existing Test Discovery

Coverage Analysis

Test Gap Analysis

Characterization Testing

AI Test Generation

Secure Test Execution

Reliability

Failure Detection

Stack Trace Analysis

Root Cause Investigation

Candidate Patch Generation

Patch Ranking

Sandbox Verification

Regression Verification

Evidence

Every major AI conclusion must contain:

Conclusion

Confidence

Evidence

Source Location where applicable

Classification

Classification:

FACT

INFERENCE

HYPOTHESIS

RECOMMENDATION

---

5. OBJECTIVE ACCEPTANCE CONTRACT

The following rule is mandatory:

No subsystem may be considered complete merely because its implementation exists.

Every difficult subsystem must have:

Implementation

+

Defined Inputs

+

Defined Outputs

+

Defined Algorithm

+

Defined Invariants

+

Defined Failure Conditions

+

Unit Tests

+

Integration Tests

+

Acceptance Tests

Claude must establish objective acceptance criteria for:

Legacy Risk

Change Safety

Hotspot Score

Repository Understanding

Patch Ranking

The exact mathematical model may be chosen by Claude, but it must be:

Explicit

Deterministic where possible

Explainable

Testable

Versioned

Reproducible

The chosen algorithm and weights must NOT remain implicit in code.

Document them.

---

6. METRIC / SCORING CONTRACT

For each scoring system, Claude must define:

Metric Name

Purpose

Input Signals

Normalization

Weights

Formula / Decision Logic

Thresholds

Output Range

Risk Categories

Evidence Mapping

Confidence Handling

Version

At minimum define objective models for:

Legacy Risk

Change Safety

Hotspot Score

Repository Understanding

Patch Ranking

For example, Claude may choose a weighted model such as:

Score =

Σ(normalized_factor × weight)

or another justified approach.

The exact model is Claude's engineering decision.

However:

The model must be explicitly documented and covered by tests.

---

7. METRIC ACCEPTANCE TESTS

For every scoring engine, create synthetic test fixtures.

The tests must verify properties such as:

Higher risk signals must not accidentally decrease risk

unless the algorithm explicitly justifies the interaction.

Higher test coverage should not increase risk without documented reasoning.

A component with high churn + high complexity + low coverage

should rank riskier than an otherwise equivalent stable component.

A well-covered, stable component should generally receive

better safety than a highly coupled, historically unstable component.

A repository with stronger evidence should have higher

understanding/confidence than one with sparse evidence.

A patch that passes more relevant verification and changes less

should generally rank better than a larger unverified patch.

Do not blindly impose these monotonic properties if the chosen mathematical model legitimately violates one.

If a property does not hold:

Document why.

Test the intended behavior.

---

8. PHASE 0 — GREENFIELD ARCHITECTURE & PLANNING

Because this is greenfield:

Do NOT perform an existing ARCHON project audit.

Instead perform a complete greenfield architecture phase.

Before significant implementation, establish:

System Architecture

Domain Model

Database Model

Repository Model

Knowledge Model

Evidence Model

Analysis Pipeline

State Machine

Artifact Storage

AI Architecture

AI Output Schemas

Sandbox Architecture

Sandbox Threat Model

Security Boundaries

Job Model

Concurrency Model

Repository Limits

API Architecture

Frontend Architecture

Testing Architecture

Deployment Architecture

---

9. REQUIRED EARLY ARCHITECTURAL DECISIONS

Claude MUST establish these early.

9.1 Database Schema

Define the database schema for at least:

Repository

RepositorySnapshot

AnalysisRun

AnalysisArtifact

Component

Dependency

Commit

Evidence

RiskAssessment

LegacyDNA

TechnicalDebtFinding

Hotspot

Assumption

ChangeAssessment

TestCase

Characterization

Execution

Failure

Investigation

Patch

PatchVerification

Incident

ModernizationRecommendation

The schema must support:

Relationships

Versioning

Traceability

Repository snapshots

Evidence

Auditability

---

10. ANALYSIS STATE MACHINE

Define a real analysis state machine.

Example conceptual states:

PENDING

QUEUED

INGESTING

SNAPSHOTTING

ANALYZING_SOURCE

ANALYZING_GIT

BUILDING_GRAPH

ARCHAEOLOGIZING

ASSESSING_RISK

ANALYZING_TESTS

CHARACTERIZING

GENERATING_TESTS

EXECUTING

INVESTIGATING

GENERATING_PATCH

VERIFYING_PATCH

COMPLETED

FAILED

CANCELLED

Claude may refine this model.

But transitions must be explicitly defined.

For each state define:

Allowed Entry

Allowed Exit

Failure Handling

Retry Behavior

Persisted State

Idempotency

No ambiguous "magic status" values.

---

11. ARTIFACT STORAGE STRATEGY

Define what belongs in:

Database

Filesystem

Object Storage abstraction

Logs

Temporary Workspace

Examples of artifacts:

Repository Metadata

Source Metadata

Dependency Graph

Analysis Results

Test Reports

Coverage

Logs

Stack Traces

Generated Tests

Generated Patches

Diffs

Excel Reports

Large/generated artifacts should not unnecessarily inflate database rows.

The architecture must support reproducibility and cleanup.

---

12. SANDBOX THREAT MODEL

Before executing repository code, define the threat model.

Treat all repository code as:

UNTRUSTED

Threats include:

Malicious setup.py

Malicious package installation

Shell commands

Filesystem access

Network access

CPU exhaustion

Memory exhaustion

Fork bombs

Infinite loops

Credential theft

Environment inspection

Container escape attempts

Malicious generated tests

Malicious generated patches

Define:

Isolation

Network Policy

CPU Limits

Memory Limits

Timeout

Filesystem Policy

Temporary Storage

Environment Variables

Secret Handling

Process Limits

Cleanup

Never execute untrusted code directly on the host.

---

13. AI STRUCTURED-OUTPUT CONTRACT

Every AI operation must use a structured schema.

Do NOT rely on unconstrained text parsing.

Define schemas for at least:

BehaviorAnalysis

HistoricalIntent

AssumptionAnalysis

RootCauseAnalysis

TestGeneration

PatchProposal

PatchRanking

ModernizationRecommendation

Each relevant schema should include:

Result

Evidence

Confidence

Reasoning Summary

Affected Components

Recommended Action

AI responses must be validated before entering the domain model.

Malformed AI output must be treated as an error.

---

14. AI HALLUCINATION CONTROL

AI must never invent repository facts.

For repository-specific claims, require evidence.

If evidence is unavailable:

UNKNOWN

or:

LOW CONFIDENCE

must be returned.

Never manufacture:

Commit

File

Function

Test

Dependency

Historical Reason

Failure

---

15. JOB / CONCURRENCY MODEL

Claude must define how long-running work executes.

The architecture must distinguish:

HTTP Request

Analysis Job

Background Execution

Sandbox Execution

AI Request

Define:

Job ID

Status

Retry Policy

Cancellation

Concurrency Limits

Duplicate Detection

Idempotency

Progress

Failure Recovery

The application must not become unusable because one large repository analysis is running.

---

16. REPOSITORY SIZE LIMITS

Define explicit initial limits for:

Maximum Repository Size

Maximum File Size

Maximum File Count

Maximum Git History

Maximum Analysis Duration

Maximum Generated Test Count

Maximum AI Context

Maximum Patch Candidates

Maximum Sandbox Runtime

These limits may be configurable.

If a repository exceeds a limit:

Reject

or

Gracefully Degrade

with a clear reason.

Never silently fail.

---

17. MVP SUPPORTED-REPOSITORY DEFINITION

Claude must explicitly define what the MVP considers a:

SUPPORTED REPOSITORY

The initial MVP should prioritize repositories that are:

Git-based

GitHub-hosted

Readable

Primarily Python

Runnable in a controlled environment

Analyzable using Python AST

Claude must define:

Supported

Partially Supported

Unsupported

repository conditions.

Do not claim multi-language support unless actually implemented.

---

18. TECHNOLOGY STACK

Use:

Backend

Python 3.12+

FastAPI

Pydantic

SQLAlchemy

Alembic

Database

PostgreSQL

SQLite for local development/testing

Source Analysis

Python AST

Tree-sitter where justified

Git

Git CLI

GitPython where useful

Graph

NetworkX

Do not introduce Neo4j without a demonstrated requirement.

Testing

pytest

coverage.py

Sandbox

Docker

Frontend

React

TypeScript

Excel

openpyxl

AI

Create:

AIProvider

with architecture supporting:

Claude

OpenAI

Local Models

---

19. TECHNOLOGY SELECTION RULE

Do not add technology simply because it sounds impressive.

Evaluate:

Necessity

Complexity

Maintenance

Security

Testing

Performance

Operational Cost

Prefer:

Simple

Reliable

Testable

Maintainable

over unnecessary complexity.

---

20. PHASE 1 — FOUNDATION & REPOSITORY INGESTION

Build:

Configuration

Logging

Database

RepositoryProvider

RepositoryContext

RepositorySnapshot

AnalysisRun

WorkspaceManager

JobManager

Repository providers:

RepositoryProvider

├— LocalRepositoryProvider

└— GitHubRepositoryProvider

---

21. GITHUB INGESTION

Workflow:

GitHub URL

↓

Validation

↓

Metadata

↓

Branch / Commit

↓

Secure Clone

↓

Snapshot

↓

RepositoryContext

Handle:

Authentication

Rate Limits

Timeouts

Retries

Pagination

Repository Not Found

Private Repository

Clone Failure

Never expose credentials.

---

22. PHASE 2 — SOURCE INTELLIGENCE

Primary language:

Python

Use Python AST.

Extract:

Files

Modules

Classes

Functions

Methods

Imports

Inheritance

Decorators

Arguments

Returns

Exceptions

Calls

Complexity

Entry Points

Tests

Configuration

Preserve source locations:

File

Line

Symbol

---

23. PHASE 3 — ARCHITECTURE & DEPENDENCY INTELLIGENCE

Reconstruct architecture using:

Source Structure

Imports

Calls

Dependencies

Framework Signals

Configuration

Build NetworkX graph.

Nodes:

Repository

File

Module

Class

Function

Test

Dependency

Configuration

Commit

Failure

Patch

Incident

Relationships:

CONTAINS

IMPORTS

CALLS

INHERITS

DEPENDS_ON

TESTED_BY

CHANGED_BY

CHANGED_WITH

FAILED_IN

FIXED_BY

AFFECTS

---

24. PHASE 4 — SOFTWARE ARCHAEOLOGY

Analyze:

Commits

Commit Messages

Authors

Dates

Changed Files

Additions

Deletions

Churn

Change Frequency

Component Age

Co-changing Files

Reconstruct:

Purpose

Inputs

Outputs

Side Effects

Dependencies

Exceptions

Callers

Called Components

Tests

Likely Invariants

Historical Changes

---

25. WHY DOES THIS CODE EXIST?

Use:

Source

Callers

Dependencies

Tests

Git History

Commit Messages

Co-changing Files

Configuration

Produce:

Likely Purpose

Historical Context

Current Role

Evidence

Confidence

Clearly distinguish:

FACT

INFERENCE

HYPOTHESIS

RECOMMENDATION

---

26. HIDDEN ASSUMPTIONS

Detect:

Null assumptions

Empty collection assumptions

Initialization assumptions

Database assumptions

Environment assumptions

External API assumptions

Ordering assumptions

Timezone assumptions

Schema assumptions

Store:

Assumption

Location

Evidence

Affected Components

Risk

Confidence

Suggested Test

---

27. PHASE 5 — LEGACY DNA

Generate Legacy DNA from:

Age

Complexity

Churn

Coupling

Coverage

Historical Failures

Hidden Assumptions

Technical Debt

Produce:

Legacy Risk

Risk Factors

Evidence

Confidence

The algorithm must be explicitly documented and tested.

---

28. TECHNICAL DEBT

Detect:

Long Functions

Large Classes

Duplicate Logic

Circular Dependencies

High Coupling

Low Cohesion

Dead Code Candidates

Deprecated APIs

Hardcoded Configuration

Broad Exception Handling

Silent Failures

Global State

Magic Numbers

Every finding:

Category

Location

Evidence

Severity

Impact

Confidence

Recommendation

---

29. HOTSPOT SCORE

Identify components where multiple risk signals overlap:

Complexity

Churn

Coupling

Coverage

Historical Failures

Hidden Assumptions

Technical Debt

Generate:

Hotspot Score

Classification

Reasons

Evidence

Possible classification:

STABLE

WATCH

RISKY

CRITICAL

The scoring algorithm must be explicit, versioned, and tested.

---

30. REPOSITORY UNDERSTANDING SCORE

Measure:

Architecture Understanding

Dependency Understanding

Behavior Understanding

Historical Understanding

Testing Understanding

Configuration Understanding

Generate:

Overall Understanding

Confidence

Evidence Coverage

Sparse evidence must reduce confidence.

The scoring model must be explicit and tested.

---

31. PHASE 6 — CHANGE SAFETY

Calculate Change Safety using:

Callers

Dependency Centrality

Test Coverage

Historical Changes

Historical Failures

Complexity

Hidden Assumptions

Coupling

Output:

Score

Risk

Factors

Evidence

Recommended Preparation

The algorithm must be:

Explicit

Explainable

Deterministic where possible

Versioned

Tested

---

32. CHANGE IMPACT

For a selected component:

Component

↓

Direct Dependents

↓

Indirect Dependents

↓

Callers

↓

Related Tests

↓

Historical Co-changes

↓

External Integrations

↓

Potential Impact

Answer:

What could break?

What depends on this?

Which tests should run?

What should happen before modification?

---

33. PHASE 7 — CHARACTERIZATION TESTING

ARCHON must distinguish:

OBSERVED BEHAVIOR

from:

CORRECT / DESIRED BEHAVIOR

Never automatically classify strange behavior as a bug.

Workflow:

Target Component

↓

Safety Analysis

↓

Safe Input Generation

↓

Execute Current Behavior

↓

Capture Output

↓

Capture Side Effects

↓

Generate Characterization Test

↓

Store Baseline

---

34. TEST GAP ANALYSIS

Analyze:

Existing Tests

Coverage

Untested Functions

Untested Branches

Missing Edge Cases

Missing Exceptions

Missing Regression Tests

Missing Characterization Tests

Prioritize using:

Risk

Legacy DNA

Change Safety

Historical Failures

---

35. AI TEST GENERATION

Provide targeted context:

Source

Existing Tests

Dependencies

Git History

Legacy DNA

Assumptions

Characterization Results

Generate:

Unit Tests

Boundary Tests

Invalid Input Tests

Exception Tests

Regression Tests

Integration Tests

Validate generated tests before execution.

---

36. PHASE 8 — SECURE EXECUTION

Use Docker or an equivalent controlled environment.

Execution:

Temporary Workspace

↓

Sandbox

↓

Test / Analysis

↓

Capture Results

↓

Cleanup

Enforce:

CPU Limits

Memory Limits

Timeout

Filesystem Restrictions

Network Restrictions

Process Restrictions

Environment Restrictions

---

37. FAILURE INVESTIGATION

When a failure occurs:

Failure

↓

Stack Trace

↓

Source

↓

Dependency Graph

↓

Git History

↓

Hidden Assumptions

↓

Legacy DNA

↓

Characterization

↓

Historical Incidents

↓

Root Cause Investigation

Produce:

Failure Summary

Root Cause Hypotheses

Evidence

Confidence

Affected Components

Recommended Verification

---

38. PHASE 9 — SELF-HEALING

Only generate repairs when sufficient evidence exists.

Workflow:

Root Cause

↓

Candidate Patches

↓

Static Validation

↓

Temporary Workspace

↓

Patch Application

↓

Testing

↓

Evaluation

---

39. MINIMAL PATCH PRINCIPLE

Prefer:

The smallest safe modification that fixes the confirmed root cause while preserving existing behavior.

Avoid unnecessary rewrites.

---

40. PATCH RANKING

Rank candidate patches using:

Correctness

Test Results

Regression Risk

Patch Size

Complexity Change

Coupling Change

Behavior Preservation

Security

The Patch Ranking algorithm must be:

Explicit

Explainable

Testable

Versioned

Do not simply select the first AI-generated patch.

---

41. PATCH VERIFICATION

A patch is VERIFIED only when:

Original Failure Fixed

AND

Characterization Tests Pass

AND

Regression Tests Pass

AND

Relevant Existing Tests Pass

AND

No New Critical Failure

AND

Patch Applies Successfully

States:

PROPOSED

TESTING

PARTIALLY_VERIFIED

VERIFIED

REJECTED

---

42. PATCH ROLLBACK

If a candidate fails:

Reject

↓

Restore Workspace

↓

Preserve Evidence

↓

Try Next Candidate

Never modify the original repository automatically.

---

43. HUMAN APPROVAL

Final repair workflow:

AI Generated Patch

↓

Sandbox Verification

↓

Human Review

↓

Approve / Reject

↓

Export / Apply

---

44. PHASE 10 — INCIDENT MEMORY

For verified repairs store:

Incident ID

Failure

Root Cause

Evidence

Affected Components

Commit

Patch

Regression Test

Verification

Confidence

Future failures may retrieve similar incidents.

Historical evidence must never override current evidence.

---

45. PHASE 11 — REPOSITORY COMPARISON

Support comparison between:

Repository @ Commit A

VS

Repository @ Commit B

Compare:

Architecture

Dependencies

Legacy DNA

Technical Debt

Risk

Coverage

Change Safety

---

46. PHASE 12 — MODERNIZATION

Analyze:

Deprecated Dependencies

Legacy APIs

High Coupling

Weak Testing

Hardcoded Configuration

Error Handling

Global State

Architecture Problems

Technical Debt

Legacy Hotspots

Generate strategies:

Add Tests

Extract Dependency

Refactor

Replace Dependency

Rewrite

For each:

Risk

Effort

Impact

Change Safety

Dependencies

Required Tests

Prerequisites

Do not automatically prefer rewrites.

---

47. API

Use FastAPI.

Expose real APIs for:

Repositories

Snapshots

Analysis Runs

Architecture

Dependencies

Archaeology

Evolution

Behavior

Understanding

Legacy DNA

Hotspots

Technical Debt

Assumptions

Change Safety

Change Impact

Characterization

Tests

Executions

Failures

Investigations

Patches

Verification

Incidents

Modernization

Reports

Use:

Validation

Pagination

Filtering

Structured Errors

OpenAPI

---

48. FRONTEND

Build a real engineering interface.

Required areas:

Repository Management

Repository Overview

Architecture

Dependency Graph

Git Evolution

Software Archaeology

Why Does This Exist?

Repository Understanding

Legacy DNA

Hotspots

Technical Debt

Change Safety

Change Impact

Characterization

Test Intelligence

Failures

Root Cause Analysis

Self-Healing

Patch Verification

Incident Memory

Modernization

Reports

No fake data.

All meaningful information must come from actual backend APIs.

---

49. EXCEL

Use:

openpyxl

Generate:

ARCHON_Legacy_Analysis.xlsx

Include:

Executive Summary

Repository Understanding

Architecture

Legacy DNA

Change Safety

Change Impact

Technical Debt

Test Gaps

Characterization

Failures

Repairs

Modernization

Software Archaeology

Incident Memory

All data must come from the same application/domain layer.

---

50. EXCEL BULK INPUT

Support:

repositories.xlsx

Columns:

Repository URL

Branch

Analysis Mode

Priority

Pipeline:

Excel

↓

Validation

↓

Analysis Jobs

↓

ARCHON Pipeline

↓

Database

↓

Results

Do not create a separate analysis engine for Excel.

---

51. OPTIONAL GITHUB WEBHOOK

Only implement after the MVP is stable.

Possible workflow:

GitHub Push

↓

Webhook

↓

Changed Components

↓

Change Impact

↓

Change Safety

↓

Targeted Tests

↓

Failure Detection

Validate webhook signatures.

Prevent duplicate event processing.

Reuse existing ARCHON engines.

---

52. SECURITY

Treat repositories and AI-generated code as untrusted.

Protect against:

Shell Injection

Path Traversal

Malicious Repository

Malicious Tests

Malicious Patches

Resource Exhaustion

Secret Leakage

Unsafe Subprocess Execution

Never expose:

AI API Keys

GitHub Tokens

Passwords

Secrets

Credentials

---

53. PERFORMANCE

Avoid unnecessary:

Repository Cloning

Git Downloads

AST Re-analysis

Graph Rebuilding

AI Calls

Test Execution

Cache using:

Repository

Commit SHA

Analysis Configuration

Engine Version

Never reuse stale results across incompatible snapshots.

---

54. ERROR HANDLING

Handle:

Invalid Repository

Repository Not Found

Private Repository

Rate Limit

Timeout

Clone Failure

Empty Repository

No Git History

No Tests

Syntax Errors

Missing Dependencies

Huge Repository

Binary Files

Circular Dependencies

Sandbox Failure

AI Failure

Malformed AI Output

Failure Not Reproduced

Intermittent Failure

Patch Failure

Regression

Interrupted Workflow

Invalid Excel

Every error should contain:

Error Code

Message

Context

Recoverability

Suggested Action

Never silently swallow errors.

---

55. OBSERVABILITY

Track:

Run ID

Repository ID

Snapshot

Commit SHA

Analysis Mode

Current Stage

Status

Start Time

End Time

Duration

Errors

AI Activity

Execution Results

Use meaningful state transitions.

---

56. TESTING REQUIREMENTS

Every major subsystem must have:

Unit Tests

Integration Tests

Acceptance Tests

Failure Tests

Edge-Case Tests

Critical subsystems require dedicated acceptance fixtures.

At minimum create acceptance tests for:

Repository Ingestion

Source Analysis

Git Analysis

Dependency Graph

Architecture Reconstruction

Legacy Risk

Hotspot Score

Repository Understanding

Change Safety

Change Impact

Characterization

Test Generation

Sandbox Execution

Failure Investigation

Patch Generation

Patch Ranking

Patch Verification

Incident Memory

---

57. END-TO-END ACCEPTANCE TEST

Create a real end-to-end test using a controlled test repository.

The test repository should contain deliberately designed characteristics such as:

Legacy Code

Multiple Modules

Dependency Relationships

Git History

Existing Tests

A Known Test Gap

A Reproducible Bug

A Fixable Root Cause

ARCHON must process it through:

Ingestion

↓

Analysis

↓

Architecture

↓

Archaeology

↓

Risk

↓

Change Safety

↓

Characterization

↓

Testing

↓

Failure

↓

Investigation

↓

Patch

↓

Verification

↓

Regression

This fixture is extremely important.

Do not rely only on arbitrary open-source repositories for automated acceptance testing.

---

58. NO FAKE FUNCTIONALITY

Never use:

Hardcoded Repository Results

Hardcoded Risk Scores

Fake Git History

Fake Test Results

Fake AI Results

Fake Patch Verification

Static Dashboard Results

Mocks are allowed only in tests.

---

59. NO DISCONNECTED MODULES

Before considering a component complete, answer:

Who calls it?

What does it consume?

What does it produce?

Where is the result stored?

Who consumes the result?

How is it tested?

How does it participate in the workflow?

If these cannot be answered:

It is not complete.

---

60. NO UNDEFINED METRICS

Do not create code like:

risk = calculate_some_score()

without defining:

What inputs?

What normalization?

What weights?

What formula?

What thresholds?

What version?

What explanation?

What tests?

Every important score must be inspectable.

---

61. NO UNVERIFIED AI

Never trust AI output simply because the model returned it.

AI output must pass:

Schema Validation

Evidence Validation

Domain Validation

Execution Validation where applicable

Generated patches must pass actual tests.

---

62. DEVELOPMENT WORKFLOW

For every phase:

UNDERSTAND

↓

DESIGN

↓

IMPLEMENT

↓

UNIT TEST

↓

INTEGRATE

↓

INTEGRATION TEST

↓

DEBUG

↓

REGRESSION TEST

↓

VERIFY

↓

COMPLETION GATE

---

63. CLAUDE EXECUTION PROTOCOL

You are authorized to create and modify project files directly.

Do not merely describe implementation.

For each phase:

1. Inspect the current implementation.

2. Identify the smallest implementation needed.

3. Design the implementation.

4. Implement it.

5. Run relevant tests.

6. Inspect actual failures.

7. Diagnose root causes.

8. Fix root causes.

9. Run regression tests.

10. Verify integration.

11. Record what was completed.

12. Continue to the next phase.

Never claim functionality is complete because code was written.

Functionality is complete only when it has been **executed and verified**.

Do not ask for permission for routine implementation decisions.

Make reasonable engineering decisions and continue.

If a major architectural or security decision is genuinely ambiguous:

Choose the simplest safe option.

Document the decision.

Continue.

Do not stop after producing a plan.

Do not stop after creating scaffolding.

Do not leave TODO/stub implementations represented as complete.

---

64. WHEN SOMETHING BREAKS

Do not work around failures.

Use:

STOP

↓

DIAGNOSE

↓

ROOT CAUSE

↓

FIX

↓

TEST

↓

INTEGRATE

↓

REGRESSION TEST

↓

CONTINUE

Do not build additional layers on top of a known broken foundation.

---

65. SCOPE MANAGEMENT

If implementation becomes complex:

Do NOT remove the MVP closed loop.

Prioritize:

1. Repository Ingestion

2. Source Analysis

3. Git Analysis

4. Dependency Graph

5. Architecture

6. Risk

7. Change Safety

8. Characterization

9. Testing

10. Failure Investigation

11. Self-Healing

12. Verification

Then:

13. Incident Memory

14. Modernization

15. Frontend Refinement

16. Excel

17. Repository Comparison

18. Webhooks

19. Advanced Multi-language Support

---

66. AGENT ARCHITECTURE

Do not create artificial agents merely to make the project appear agentic.

Potential logical roles:

Archaeologist

Testing Agent

Investigation Agent

Repair Agent

Modernization Agent

Documentation Agent

But deterministic work remains deterministic.

The agents must operate over shared repository knowledge.

Use orchestration rather than disconnected autonomous modules.

---

67. FINAL SYSTEM FLOW

The final ARCHON pipeline should be:

REAL GITHUB REPOSITORY

↓

INGESTION

↓

REPOSITORY SNAPSHOT

↓

SOURCE INTELLIGENCE

↓

ARCHITECTURE RECONSTRUCTION

↓

DEPENDENCY KNOWLEDGE

↓

GIT ARCHAEOLOGY

↓

BEHAVIOR ANALYSIS

↓

"WHY DOES THIS EXIST?"

↓

HIDDEN ASSUMPTIONS

↓

REPOSITORY UNDERSTANDING

↓

LEGACY DNA

↓

TECHNICAL DEBT

↓

HOTSPOTS

↓

CHANGE SAFETY

↓

CHANGE IMPACT

↓

CHARACTERIZATION

↓

TEST ANALYSIS

↓

TEST GENERATION

↓

SECURE EXECUTION

↓

FAILURE

↓

ROOT CAUSE ANALYSIS

↓

CANDIDATE PATCHES

↓

PATCH RANKING

↓

SANDBOX VERIFICATION

↓

REGRESSION TESTING

↓

VERIFIED / REJECTED

↓

INCIDENT MEMORY

↓

MODERNIZATION

---

68. FINAL ACCEPTANCE CRITERIA

ARCHON is NOT complete because:

Frontend loads

API returns 200

AI generates text

Graphs render

Excel downloads

Tests exist

Docker starts

Repository clones

ARCHON is complete only when:

REAL REPOSITORY

↓

REAL ANALYSIS

↓

REAL EVIDENCE

↓

REAL RISK

↓

REAL TESTING

↓

REAL FAILURE INVESTIGATION

↓

REAL PATCH

↓

REAL SANDBOX VERIFICATION

↓

REAL REGRESSION VERIFICATION

works end-to-end.

---

69. FINAL ENGINEERING PRINCIPLES

PRINCIPLE 1

Do not optimize for writing code quickly.

PRINCIPLE 2

Optimize for producing the largest amount of genuinely working, tested, integrated functionality.

PRINCIPLE 3

Do not optimize for the number of agents.

PRINCIPLE 4

Do not optimize for the number of technologies.

PRINCIPLE 5

Do not optimize for code volume.

PRINCIPLE 6

Prefer deterministic engineering where AI is unnecessary.

PRINCIPLE 7

Use AI where reasoning genuinely provides value.

PRINCIPLE 8

Every important AI conclusion must have evidence and confidence.

PRINCIPLE 9

Every important metric must have an explicit algorithm and acceptance tests.

PRINCIPLE 10

Never trust an AI-generated patch until it has been verified.

PRINCIPLE 11

Never automatically modify the original repository.

PRINCIPLE 12

Prefer minimal safe patches over unnecessary rewrites.

PRINCIPLE 13

Characterize strange legacy behavior before changing it.

PRINCIPLE 14

Observed behavior is not automatically correct behavior.

PRINCIPLE 15

Low confidence means more investigation, not fabricated certainty.

PRINCIPLE 16

Every phase must be tested before proceeding.

PRINCIPLE 17

Integration is more important than feature count.

PRINCIPLE 18

Do not stop at planning.

PRINCIPLE 19

Do not stop at scaffolding.

PRINCIPLE 20

Never declare functionality complete until it has actually executed and been verified.

---

70. START NOW

This is a **GREENFIELD PROJECT**.

There is no existing ARCHON codebase.

You are authorized to create and modify the project files directly.

Start with:

PHASE 0

GREENFIELD ARCHITECTURE & PLANNING

During Phase 0:

1. Understand the complete specification.

2. Define the architecture.

3. Define the domain model.

4. Define the database schema.

5. Define the analysis state machine.

6. Define artifact storage.

7. Define the sandbox threat model.

8. Define AI structured-output schemas.

9. Define the job/concurrency model.

10. Define repository limits.

11. Define the MVP supported-repository contract.

12. Define all important scoring algorithms.

13. Define acceptance tests.

14. Create the implementation roadmap.

15. Create the initial project structure.

Then immediately begin implementation.

Do not spend the entire process planning.

For every phase:

IMPLEMENT

↓

TEST

↓

INTEGRATE

↓

VERIFY

Then continue.

ULTIMATE RULE

Do not optimize for writing code quickly. Optimize for producing the largest amount of genuinely working, tested, integrated functionality within the available scope.

Do not build ARCHON as a collection of disconnected modules. Build it as one evidence-driven software engineering system where repository understanding feeds risk analysis, risk analysis feeds testing, testing feeds failure investigation, failure investigation feeds self-healing, verification feeds regression protection, and accumulated knowledge feeds modernization.

Every difficult subsystem must have objective acceptance criteria. Every important metric must have an explicit, explainable, versioned algorithm. Every AI conclusion must be evidence-backed. Every generated patch must be executed and verified.

You are building the actual product. Start with Phase 0, establish the engineering contract, create the plan, and then execute it.